- *Access token*—Access tokens are used to authorize access to a resource. These are the tokens typically referred to when you talk about bearer authentication. They come in two flavors:
    - *Self-contained*—These are the most common tokens, with JWT as the most common format. They contain metadata, claims, and a signature. The strength of self-contained tokens—that they contain all the data and can be validated offline—is also their weakness, as they can't be revoked. Due to this, they typically have a limited valid lifespan. They can also become large if they contain many claims, which increases request sizes.
    - *Reference token*—These don't contain any data and are typically a random string. When a protected API receives a reference token, it must exchange the reference token with the identity provider for the claims (for example, a JWT). This approach ensures more privacy, as the claims are never exposed to the client, and the token can be revoked at the identity provider. However, it requires an extra HTTP round trip every time the API receives a request. This makes reference tokens a good option for high-security environments, where the performance effect is less critical.
- *ID token*—This token is used in OpenID Connect (http://mng.bz/a1M7) to describe an authentication event. It may contain additional claims about the authenticated user, but this is not required; if the claims aren't provided in the ID token, they can be retrieved from the identity provider's UserInfo endpoint. The ID token is *always* a JWT, but you should never send it to other APIs; it is not an access token. The ID token can also be used to log out the user at the identity provider.
- *Refresh token*—For security reasons, access tokens typically have relatively short lifetimes, sometimes as low as 5 minutes. After this time, the access token is no longer valid, and you need to retrieve a new one. Making users log in to their identity provider every 5 minutes is clearly a bad experience, so as part of the OAuth or OpenID Connect flow you can also request a *refresh token*.

  When an access token expires, you can send the refresh token to an identity provider, and it returns a new access token without the user's needing to log in again. The power to obtain valid access tokens means that it's critical to protect refresh tokens; should an attacker obtain a refresh token, they effectively have the power to impersonate a user.

In most of your work building and interacting with APIs, you'll likely be using self-contained JWT access tokens. These are what I'm primarily referring to in this chapter whenever I